Formalization of Gödel's Incompleteness Theorems in Basic Recursive Arithmetic (BRA)

Summary of an Agda development

May 2026

Abstract

We present a fully formalized proof of both Gödel's First and Second Incompleteness Theorems in a tree-based variant of Guard's Basic Recursive Arithmetic (BRA), executed in Agda 2.8 with `--safe --without-K --exact-split`. The development contains **zero postulates and zero holes**; every per-file solo typecheck completes in under one second (with one historical exception, the 16k-line dispatcher cascade for *thmT*).

The headline results are:

- *thm11* : *Deriv G* $\to$ *Deriv* $\bot$ (Gödel I: provability of the diagonal sentence implies $\bot$);
- *godelII* : *Deriv Con* $\to$ *Deriv* $\bot$ (Gödel II: provability of consistency implies $\bot$).

The development follows Guard's 1963 lecture notes on recursive arithmetic exclusively. The intermediate construction is a single first-order computable function *constr5* : *Fun1* realizing Guard's "$F(x)$" (page 17 of his notes); each step in the chain is given by an explicit equational derivation in BRA.

1 Introduction

The aim of the project is to give a feasible, ASCII-only, syntax-first Agda formalization of Gödel's two incompleteness theorems, following Guard's 1963 system of *Basic Recursive Arithmetic* [1]. The intent is to avoid historical baggage (Gödel numbering of Hilbert proofs, primitive recursive arithmetic with explicit numerals, "encoded" substitution as a Hilbert-system meta theorem) by working inside a small tree-based equational system from the outset:

- Terms are built from O (leaf), *var* n, $ap_1\ f\ t$, $ap_2\ g\ t_1\ t_2$.
- Function symbols come in two arities: Fun_1 and Fun_2, with primitive constants $\{I, Fst, Snd, Comp, Comp_2, Z$ and $\{Pair, Const, Lift, Post, Fan, IfLf, TreeEq, treeRec\}$.
- Equations are objects *eqn* $\ell\ r$ between terms; formulas *Formula* extend equations with *not* and *imp*.
- Provability is an inductive type *Deriv F* with constructors for the algebraic axioms (ax_*), the equality axioms, the propositional axioms (ax_K, ax_S, ax_{Neg}, $ax_{ExFalso}$, $ax_{Contrapos}$), modus ponens (*mp*), substitution (*ruleInst*), and binary tree induction (*ruleIndBT*).

The natural numbers are represented as binary trees (*lf* for 0, *nd lf n* for the successor in unary). The encoding *reify* : *Tree* $\to$ *Term* maps *lf* $\mapsto O$ and *nd a b* $\mapsto ap_2\ Pair\ (reify\ a)\ (reify\ b)$.

2 The provability predicate $thmT$

The Gödel-style "*th*" provability predicate is realized in BRA by a single Fun_1-functor:

$$thmT \;=\; Rec\ stepProtoWrapped \;:\; Fun_1.$$

Here *stepProtoWrapped* is a Fun_2 assembled from a 40-deep linear *IfLf* cascade (`BRA/Thm/StepProto.agda` and `BRA/Thm/ThmT.agda`, ∼16k lines), one branch per primitive proof rule: ten equational axioms of Group I (`axI`, `axFst`, …, `axKT`), eight Group II axioms (`axIfLfL`, `axIfLfN`, four `axTreeEq*`, `axGoodstein`), nine equality and propositional rules, and three rules of inference (*mp*, *ruleInst*, *ruleIndBT*).

The *key* property of $thmT$ is that on the encoded form of any proof tree it returns the encoded conclusion of that proof. This is the content of the *encoding-soundness* lemma *encode* : $(P : Formula) \to Deriv\ P \to \Sigma\ Tree\ (\lambda y \to Deriv\ (thmT\ (reify\ y) = reify\ (codeFormula\ P)))$, proved by induction on the *Deriv* derivation in `BRA/Thm/ThmT.agda`.

2.1 Self-substitution closure

A second key property is that $thmT$ contains no free variable (it is a closed combinator):

$$thClosed : \forall x\ r, \quad substF_1\ x\ r\ thmT = thmT$$

and that the encoding of $thmT$ is var-free:

$$codeF1Th\_noVar : \forall x\ repl, \quad codedSubst\ repl\ (code\ (var\ x))\ (codeF_1\ thmT) = codeF_1\ thmT.$$

Both are `refl` inside the abstract block where $thmT$ unfolds.

3 Gödel's First Incompleteness Theorem (Theorem 11)

3.1 The diagonal sentence

Following Guard, define

$$\begin{aligned} F_{\text{pre}} &:= \neg\big(thmT\ (v_1) \;=\; sub\ (v_0)\ (v_0)\big), \\ j &:= codeFormula\ F_{\text{pre}} \quad \text{(a closed Tree)}, \\ G &:= F_{\text{pre}}[v_0 := reify\ j] \;=\; \neg\big(thmT\ (v_1) \;=\; sub\ i\ i\big), \end{aligned}$$

where $i := reify\ j$ and *sub* is the BRA-internal coded substitution function (`BRA/Sub.agda`). The sentence G has v_1 free; informally, G asserts of itself that no proof tree v_1 proves the formula whose code is $j[v_0 := i]$.

3.2 Theorem 11

Pulling apart the abstract closure (`BRA/Thm11Abstract.agda`), the precise statement is:

Theorem 1 (`thm11`, Gödel I)**.** *thm11* : $Deriv\ G \to Deriv\ \bot$, *where* $\bot := atomic\ (eqn\ trueT\ falseT)$ *with* $trueT := O$ *and* $falseT := ap_2\ Pair\ O\ O$.

Sketch. From any $\pi : Deriv\ G$ extract its encoding (y_d, E_1) where $E_1 : Deriv\ (thmT\ (reify\ y_d) = reify\ (codeFormula\ G))$. A separate *diagonal lemma* `diagonal` (`BRA/Thm11Diagonal.agda`) yields $N : Deriv\ \neg(thmT\ (reify\ y_d) = reify\ (codeFormula\ G))$ from π, and $ax_{ExFalso}$ + two *mp*'s collapse E_1 and N to $Deriv\ \bot$. □

The diagonal lemma is constructed inside an abstract block in `BRA/Thm11Diagonal.agda` using *thClosed*, *codeF1Th_noVar*, and *cc* (the local instance of the meta-equation *codedSubst* (*code i*) (*code* v_0) j = *codeFormula* G, which is provable by induction on F_{pre} via *codeCommutesFormula* in `BRA/CodeCommutes.agda`).

4 Gödel's Second Incompleteness Theorem (Theorem 14)

The second theorem is the more substantial construction. It says that the consistency formula *Con* entails $\bot$:

Theorem 2 (`godelII`, Gödel II)**.** *godelII* : *Deriv Con* $\to$ *Deriv* $\bot$, *where Con* := $\neg\big(thmT\ (v_0) = reify\ (codeFormula\ \bot)\big)$ *(consistency: "no proof, encoded as* v_0*, derives* $\bot$*").*

The construction follows Guard's pages 16–17 step by step.

4.1 Theorem 12 — the encoded "*th f*" clause

For every Fun_1-functor f, Theorem 12 produces a Fun_1 D_f such that under the canonical encoding, *thmT* evaluated at $D_f\ x$ is provably equal to the encoded form of "$f(num\ x) = num\ (f\ x)$". Formally:

Theorem 3 (*thm12_F*$_1$, parametric Theorem 12 for Fun_1)**.** *thm12_F*$_1$: $(f : Fun_1) \to$ *Thm12_F*$_1$*_Result f where the record Thm12_F*$_1$*_Result f packages a* $D_f : Fun_1$ *together with proof* : $(x : Term) \to$ *Deriv* $(thmT\ (D_f\ x) = codeFTeq_1\ f\ x)$*. The conclusion* $codeFTeq_1\ f\ x$ *is the literal Pair-shape encoded equation* $ap_1\ f\ (cor\ x) = cor\ (f\ x)$*.*

(Analogous statement for Fun_2.) This is a structural recursion on f: there is one clause per Fun_1 / Fun_2 constructor (15 total), each in its own `BRA/Thm12/Parts/*.agda` file. The composite cases (*Comp*, $Comp_2$, *Lift*, *Post*, *Fan*, *treeRec*) dispatch to per-case modules; the recursive *TreeEq* case uses `ruleIndBT2` (2-tree-pair induction with diagonal cross-IHs) to establish closure. The whole theorem is in `BRA/Thm12.agda`, module `FromBridges`, and contains zero postulates / zero holes.

4.2 Theorem 13 — under hypothesis "$f\ x = y$"

Building on Theorem 12, one produces under the hypothesis $f\ x = y$ the encoded form of "$f(num\ x) = num\ y$":

Lemma 4 (*thm13_F*$_1$)**.** *For every* $f : Fun_1$ *and every Theorem 12 result* r_{12} *for f, thm13_F*$_1$ $f\ r_{12}\ x\ y$: *Deriv* $(f\ x = y) \to$ *Deriv* $(thmT\ (D_f\ x) = codeFTeq_{1,\text{Hyp}}\ f\ x\ y)$ *where the conclusion replaces the "cor* $(f\ x)$*" slot of* $codeFTeq_1$ *by cor y. One line: ruleTrans on* r_{12}*.proof x with* $cong_R$ *Pair on* $cong_1$ *cor of the hypothesis.*

4.3 Theorem 14 — the consistency-to-bot construction

This is the heart of Gödel II. The contract abstracted in `BRA/Thm14Abstract.agda` asks for:

1. a Fun_1-functor *constr5* such that $thmT\ (constr5\ x)$ syntactically encodes $\bot$;
2. a parametric internal-implication

   $step5 : (x : Term) \to Deriv\big((thmT\ x = reify\ (codeFormula\ G))\ imp\ (thmT\ (constr5\ x) = reify\ (codeFormul$

The closure *closureToG* then applies $ax_{Contrapos}$ to *step5* instantiated at v_1, *ruleInst*s *Con* at *constr5* v_1, modus-ponenses to obtain $\neg(thmT\ v_1 = i)$, transports through *subIIeqcG* (*sub i i* = *cor G*) and G_{unfold} (propositional normal form of G), arriving at *Deriv G*. Composing with *thm11* gives *godelII* : *Deriv Con* → *Deriv* ⊥.

4.3.1 Construction of *constr5*

Following Guard's page 17, the construction relies on two pre-existing tautologies, encoded:

- $t := (v_0 = v_2)\ imp\ ((v_1 = v_2)\ imp\ (v_0 = v_1))$ (transitivity);
- $t' := (v_0 = v_1)\ imp\ ((\neg(v_0 = v_1))\ imp\ \bot)$ (ex falso).

Both are derivable in BRA (`BRA/Thm14Numerals.agda`), and via *encode* they yield closed Trees $tterm, t'term$ such that $thmT\ (reify\ tterm) = reify\ (codeFormula\ t)$ (and analogously for t').

The chain is:

$$\begin{aligned}
f_part(x) &:= ruleInst\ v_0\ (th\ \hat{x})\ \big(ruleInst\ v_1\ (sub\ i\ i)\ (ruleInst\ v_2\ cor\ G\ tterm)\big),\\
g_part(x) &:= mp\big(mp(f_part(x), D_{thmT}\ x), D_{sub}\ i\ i\big),\\
K_part(x) &:= ruleInst\ v_1\ (cor\ x)\ x,\\
h_part_skel(x) &:= ruleInst\ v_0\ (th\ \hat{x})\ \big(ruleInst\ v_1\ (sub\ i\ i)\ t'term\big),\\
constr5_{\text{final}}(x) &:= mp\big(mp(h_part_skel(x), g_part(x)), K_part(x)\big).
\end{aligned}$$

The substituents are the encoded mixed-form Pairs of Guard: $\widehat{th\,\hat{x}} := ap_2\ Pair\ (reify\ tagAp_1)\ (ap_2\ Pair\ (reify\ (code$ $\widehat{sub_{i\,i}} := reify\ (code\ (ap_2\ sub\ i\ i))$, and $cor\ G := reify\ (code\ (reify\ (codeFormula\ G)))$.

4.3.2 The five steps and their lemmas

Following Guard's enumeration (`guard15.pdf` pages 16–17):

Step 1 — *step1_l* (`Th14Constr5.agda`). By Theorem 13 applied to *thmT* at $(x, cor\ G)$ under hypothesis $thmT\ x = cor\ G$, get $Deriv\ (PrAtX\ x\ imp\ thmT\ (D_{thmT}\ x) = codeFTeq_{1,\text{Hyp}}\ thmT\ x\ cor\ G)$.

Step 2 — *step2_l* (`Th14Step2.agda`). By Theorem 13 applied to *sub* at $(i, i, cor\ G)$ under closed hypothesis *subIIeqcG*.

Step 3 — *step3_l* **(= *g_part_l*)** (`Th14Step3.agda`, `Th14Step3Canon.agda`). Three sb-applications on t via *thmTSubLemma*, canonicalized via *subT_codeFormula_imp/atomic* and leaf preservation lemmas, then two *mp* dispatches at *g_part_inner* and *g_part*. Output: $Deriv\ (PrAtX\ x\ imp\ thmT\ (g\_part\ x) = th\ \widehat{\hat{x} = sub\ i\ i})$.

Step 4 — *K_part_l* (`Th14Step4Final.agda`). One sb-application at v_1 on the diagonal sentence G_{norm} via *thmTSubLemma* + the $PrAtX\ x$ hypothesis to swap $thmT\ x \mapsto cor\ G$, canonicalized via *subT_codeFormula_neg/atomic* and leaf preservation. Output: $Deriv\ (PrAtX\ x\ imp\ thmT\ (K\_part\ x) = th\ \widehat{\hat{x} \neq sub\ i\ i})$.

Step 5 — *step5_l* (`Th14Step5.agda`). Two outer *mp* dispatches at *constr5*_inner, final and $constr5_{\text{final}}$, with *step3_l* feeding the inner antecedent and *K_part_l* standing in syntactically as the second-mp witness. The dispatcher's syntactic projection of the consequent gives the encoded ⊥. Output: $Deriv\ (PrAtX\ x\ imp\ thmT\ (constr5_{\text{final}}\ x) = reify\ (codeFormula\ \bot))$.

Auxiliary — *h_part_skel_l* (`Th14Step5HPart.agda`). Two sb-applications on t', mirror of step 3 (one fewer layer because t' has only two free variables). Output: *thmT* (*h_part_skel x*) = ex $\widehat{\text{falso form}}$. Used as *step5_l*'s inner d_1.

The headline composition (`BRA/GoedelIIFull.agda`) is then:

```
r12_th  : Thm12_F1_Result thmT
r12_th  = FromBridges.thm12_F1 thmT

r12_sub : Thm12_F2_Result sub
r12_sub = FromBridges.thm12_F2 sub

module Step5 = BRA.Th14Step5.Th14Step5 r12_th r12_sub
module Final = BRA.GoedelII.Compose Step5.constr5_final Step5.step5_l

godelII : Deriv Con -> Deriv bot
godelII = Final.godelII
```

5 Methodology

5.1 Fast typecheck as a correctness oracle

Every per-file solo typecheck runs in well under one second (with the historical exception of `BRA/Thm/ThmT.agda` at ∼19s; this is the 16k-line dispatcher cascade and was already in place before the Gödel II push). The session followed a strict rule: *slow typecheck = mathematical mismatch, stop and re-architect*. In practice this prevented two specific traps:

1. Trying to feed Agda an opaque normalisation it cannot perform (e.g. unfolding *codeFormula G* at a parametric substituent position). The fix is always: add a single targeted refl-witnessable lemma inside the right `abstract` block (where the opaque definition unfolds), and use it as a black box outside.

2. Forcing structural recursion through opaque trees (*codedSubst*/*codedSubstT* on $codeF_1\ thmT$, etc.). Same fix: prove the equation once inside the abstract block, expose only the type signature outside.

5.2 Architectural choice for *constr5*

The original `Th14Constr5Final.constr5` wraps *h_part* with two *additional* encoded-mp layers (*h_part_thxLayer*, *h_part*). Under the *thmT* dispatcher these would consume both antecedents of *h_part_skel*'s ex-falso shape, collapsing the value of *thmT* (*h_part x*) to $\bot$ before it can serve as antecedent to *constr5*_inner. The fix is to use *h_part_skel* directly in $constr5_{\text{final}}$ (pure sb-chain, no internal mp's), matching Guard's intent on page 17 and the comment in `Th14Constr5.agda`. The two outer *mp*'s in $constr5_{\text{final}}$ then consume the two antecedents exactly as Guard's chain prescribes.

6 Statistics

6.1 File-by-file summary of the Gödel II push

File	LoC	Solo typecheck
`BRA/Th14SubTLeaf.agda`	374	0.18 s
`BRA/Th14Step3Canon.agda`	582	0.60 s
`BRA/Th14Step3.agda`	633	0.81 s
`BRA/Th14Step4Final.agda`	213	0.84 s
`BRA/Th14Step5HPart.agda`	394	0.84 s
`BRA/Th14Step5.agda`	305	0.76 s
`BRA/GoedelIIFull.agda`	50	0.71 s

6.2 Earlier infrastructure (already in place)

- `BRA/Thm/ThmT.agda`: 16k LoC, 40-deep dispatcher cascade, 19 s.
- `BRA/Thm12.agda`: structural Theorem 12 with one Parts file per constructor, 0.65 s.
- `BRA/Thm13.agda`: ∼80 LoC, 0.1 s.
- `BRA/Thm11.agda`, `Thm11Abstract`, `Thm11Diagonal`: ∼300 LoC total.
- `BRA/Thm14Abstract.agda`: ∼250 LoC, the *closureToG* machinery.
- `BRA/Thm14Constr5Final.agda`, `Th14StepHPre`, `Th14HUnfolds`, `Th14GPartUnfolds`: ∼700 LoC of Pair-of-Comp2 unfold lemmas.

6.3 Constraints maintained throughout

- `--safe --without-K --exact-split` on every file.
- ASCII only. No Unicode.
- No postulates. No holes. No `with`-abstraction. No dot patterns.
- No new *Deriv* axioms beyond Guard's primitive list.

7 Discussion

The two Gödel theorems in BRA, side by side:

- **Gödel I.** If the diagonal sentence G is provable, then $\bot$ is provable. Equivalently (under the meta-assumption that BRA is consistent): G is unprovable.
- **Gödel II.** If the consistency formula *Con* is provable, then $\bot$ is provable. Equivalently: BRA cannot prove its own consistency.

The intermediate functions $constr5_{\text{final}}$ and the parametric step lemmas *step1_l*–*step5_l*, *K_part_l*, *h_part_skel_l* are *first-order* in the sense of recursive-arithmetic definability: each is a closed combinator built from primitive Fun_1 / Fun_2 symbols, and each *Deriv* derivation is given by an explicit equational proof in the BRA system. No external soundness theorem is invoked.

7.1 Comparison with prior formalisations

To our knowledge the only previous machine formalisation of Gödel's *Second* Incompleteness Theorem is Paulson's Isabelle/HOL development of hereditarily finite set theory [2]. Other formalisations (O'Connor in Coq, Han in Lean, Carneiro's metamath work) target Gödel's *First* theorem only.

Paulson's development uses HF set theory and full Gödel numbering of Hilbert-style proofs, with encoded substitution treated as an external soundness theorem. The present BRA development by contrast follows Guard [1]: the "provability predicate" $thmT$ is itself a primitive-recursive combinator built from $Rec + Fan + Lift + IfLf$; encoded substitution is performed by an explicit primitive recursive function $sub : Fun_2$; and the entire chain (Gödel I and II) stays first-order, no Hilbert-proof Gödel numbering, no external soundness theorem.

References

[1] J. R. Guard, *Lecture notes on recursive arithmetic*, Applied Mathematics Division, Argonne National Laboratory, 1963.

[2] L. C. Paulson, *A machine-assisted proof of Gödel's incompleteness theorems for the theory of hereditarily finite sets*, Review of Symbolic Logic, vol. 7 no. 3, 2014, pp. 484–498.